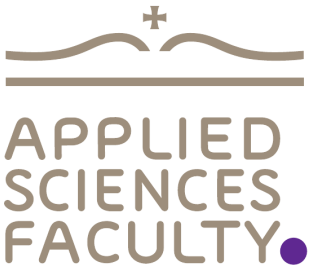
C++ — лекція 20. std::string (Draft)





©Олег Фаренюк,
Creative Commons - Attribution Share Alike (CC BY-SA)

C++ включає значно просунутіші засоби роботи із текстовим стрічками, ніж C-стрічки, які ми вже розглядали.

Реалізуються вони у файлі string:

```
#include <string>
```

Головний клас — string. Насправді, це шаблон, інстанційований для типу char:

```
typedef basic_string<char> string;
```

але для нас це поки несуттєво.

Увага! Огляд нижче не претендує на повноту! Його ціль — лише продемонструвати основні можливості стрічок. Детальніше див., наприклад, `http://www.cplusplus.com/reference/string/string/`

- Цей клас має конструктор за замовчуванням і конструктор, що приймає стрічку-літерал, з можливістю вказувати, скільки символів копіювати, можливість створити стрічку із вказаної кількості повторів символу, [1] :

```
string empty;
string str1("ABC123 cde");
char cs[] = "ABC123 cde";
string str1a(cs,3); // скопіює "ABC"
string str2(17, '!');
```

- Існують конструктори копіювання та присвоєння.

```
string s1("KLM"),s2;
s2 = s1;
string s3(s1);
```

Під час конструювання копіюванням можна вказувати, з якого символу починати і скільки символів копіювати. (Якщо вказано більше, ніж є, буде скопійовано всі наявні.)

```
string s4("KLMN");
string s5(s4,1,2); // Скопіює "LM"
```

◇ Якщо передати nullptr в ролі C-стрічки – невизначена поведінка до C++23, не компіюється після. Однак, конструювати з nullptr з вказанням розміру – все ще UB.

[1] Детальніше — див. документацію.

Доступ до окремих літер

- Можна звертатися до окремих літер як до елементів масиву[2]:

  ```
  cout << str1[5] << endl;
  ```

  Нумерація — з нуля. Контролю виходу за межі не здійснюється! — operator[] оптимізовано для досягнення максимальної швидкодії.
- Розмір стрічки можна визначити за допомогою методу length() або його синоніму size():

  ```
  cout << str1.length() << endl;
  ```

  Він показує реальну кількість символів, без всіляких нульових символів у кінці.
- Якщо бажано контролювати, чи не виходимо за межі стрічки, можна використати метод at():

  ```
  char a = str1.at(2);
  ```

- Якщо відбувається вихід за діапазон, at() генерує виключення `out_of_range`.

[2] Перевизначено оператор [].

- Перевірити, чи стрічка порожня, можна трішки швидше, ніж:

```
if ( str1.length()==0 )
   { ... }
```

Для цього є спеціальний метод, empty():

```
if ( str1.empty() )
{ ...      }
```

- Метод clear() очищає стрічку.
- Для вказання "до кінця стрічки" під час копіювання та "не знайдено" під час пошуку, використовується статична константа npos, оголошена в класі string:

```
static const size_t npos = -1;
```

Тип size_t — беззнаковий, тому -1 означає найбільше додатне число. Приклад:

```
string s4("KLMNOPQ");
string s5(s4,3,2); // Скопіює "MNOPQ"
```

- Оператор присвоєння перевантажено для char, тому можна писати такі конструкції:

```
str += 'a'; // Додаємо літеру в кінці стрічки
```

Якщо потрібно із std::string добути звичайну C-стрічку, слід скористатися методом c_str():

```
const char *cprt = str1.c_str();
```

УВАГА! Змінювати таку стрічку не можна, тому присвоювати її можна тільки "const char *".

УВАГА! Будь-які операції, що змінюють вихідну стрічку, можуть зробити вказівник, отриманий таким чином, невірним! Тому викликати c_str() слід безпосередньо перед використанням.

Призначення:

- Взаємодія із кодом на C, який інших стрічок не знає.
- На жаль, файлові потоки не вміють приймати в ролі аргументів std::string, тому доводиться робити так:

```
string filename;
.....
ofstream file ( filename.c_str() );
```

Порівняння стрічок

- Для порівняння стрічок використовується сімейство методів compare.
- Порівнювати об'єкти типу std::string можна як із собою, так і з C-стрічками.
- compare порівнюють стрічки по літерах, повертають 0, якщо стрічки рівні, від'ємне число, якщо перший відмінний символ менший у першій із порівнюваних стрічок, або всі наявні символи збігаються, але перша стрічка коротша, додатне — якщо навпаки.
- Найпростіший варіант використання compare:

```
string str2("abce1");
string str3("abce");

if( str2.compare(str3) == 0 )
{
    cout << "str2 and str3 are equal." << endl;
}
```

Більш просунуті compare

Крім compare, що порівнює дві цілі стрічки, є варіанти для порівняння стрічок, починаючи із певного номера, і порівнювати задану кількість символів. Для стрічок:

```
string  str2("abce1");
string  str3("abce");
```

Команда:

```
cout << ( str2.compare(0, 4,  str3)==0 ? "Equal":"Not equal"  );
```

Надрукує "Equal", так як 4 символи, починаючи з нульового, у них рівні, а:

```
cout << ( str2.compare(0, 5,  str3)==0 ? "Equal":"Not equal"  );
```

Надрукує "Not equal".

Є ще й інші варіанти. Детальніше — див. документацію, або книги по C++

Конкатенація:

Стрічки можна об'єднувати:

```
string  str4("1234");
string  str5("ABCD");
string  str6;

str6 = str4+str5;
cout << str6 << endl;

str4 += str5;
cout << str4 << endl;

str6 += " _!";
cout << str6 << endl;
```

Вставка

Можна вставляти одну стрічку в іншу:

```
cout << str5 << endl; // "ABCD"
str5.insert(2,"__");
cout << str5 << endl; // "AB__CD"
```

Зміна розміру

- Метод resize() змінює розмір стрічки. Якщо розмір збільшується, нові символи заповнюються вказаним символом, або символом за замовчуванням:

```
string s("zrsdrsdzdszrb");
s.resize(30,' ');
s.resize(50);
```

- Для оптимізації реалокувань, підтримується також метод reserve().
- Поточний "запас" можна побачити методом capacity().
- Як резервується на практиці, див., наприклад: `https://lemire.me/blog/2023/10/23/appending-to-an-stdstring-character-by-character-how-does-the-ca`

Пошук

- Є ціле сімейство методів пошуку у стрічці.
- Найпростіший, **find** — пошук першого співпадіння. Повертає позицію **першого** співпадіння, або значення **string::npos**, яке в цьому випадку означає "*не знайдено*":

```
string  str7("We have apple");
if( str7.find("apple")==string::npos )
{
    cout << "Apple not found." << endl;
}
```

- **rfind** — пошук останнього співпадіння.
- **find_first_of** — пошук першого входження будь-якої літери із списку.
- **find_last_of** — аналогічно, останнього.
- **find_first_not_of** і **find_last_not_of** — пошук першого чи останнього не-входження.

Виділення, заміна та видалення підстрічки

- Метод substr() дозволяє створити стрічку із частини іншої стрічки:

```
string  str7("We have apple");
string  str7a;
str7a = str7.substr(  0,7);  // str7a = "We have"
```

- Методи сімейства replace замінюють підстрічку. Детальний список – див. документацію. Приклад:

```
string  str7("We have apple");
str7.replace(  str7.find("apple"),5,  "table");
```

Цей код замінить слово "apple", початок якого знайдено за допомогою методу find на "table".

- Методи erase() видаляють підстрічки:

```
string  str8("We do not have apple");
str7.erase(  6,  4  );    // Видаляємо "not "
```

Обмін стрічок вмістом

Швидко (за константний час) обміняти вміст двох стрічок можна за допомогою swap:

```
string  str8("We do not have apple"),
string  str9("We have a tensor");
swap(str8, str9);
```

C++17 та C++20

- C++17 додав оператор перетворення в std::string_view.
- C++20 додав методи: starts_with() та ends_with().
- C++23 додав метод contains().
- Ще один цікавий метод з C++23: resize_and_overwrite() – змінити розмір, якщо потрібно додати нові елементи, вони будуть згенеровані викликом callable: `https://en.cppreference.com/w/cpp/string/basic_string/resize_and_overwrite`.
- Див. також: `https://en.cppreference.com/w/cpp/string/basic_string/replace_with_range`.

Посилання

- `https://devblogs.microsoft.com/oldnewthing/20240510-00/?p=109742` – "An informal comparison of the three major implementations of std::string", про різні компроміси реалізацій, зокрема і щодо SSO.
- `https://devblogs.microsoft.com/oldnewthing/20230803-00/?p=108532` – "Inside STL: The string".

C++ включає ряд засобів роботи із стрічками, успадковані з C.

Наприклад, порівняння двох C-стрічок можна здійснити за допомогою функції strcmp(), схожої за поведінкою на compare:

```
char *cstr1="ABCD";
char *cstr2="ABCDE";

cout << ( strcmp(cstr1,cstr2)==0 ? "Equal":"Not equal" );
```

Знайти розмір стрічки за допомогою strlen() і т.д. Детальніше ми їх розглядати не будемо.

Перетворення стрічок в числа і назад

Стандартна бібліотека C та C++ містить наступні корисні функції:

double atof (**const char*** str); // Перетворити Сстрічку— в *double*
int atoi (**const char** * str); // Перетворити стрічку в ціле число
long int atol (**const char** * str); // ——— в *long int*

double strtod (**const char*** str, **char**** endptr);
// Перетворює стрічку в *double*,
// якщо *endptr* не *0*, зберігає у змінну за цією адресою
// вказівник на перший символ, який не брав участі в перетворенні ———
// перший символ після числа

long int strtol (**const char*** str, **char**** endptr, **int** base);
// Аналогічно для перетворення в *long int*,
// *base* вказує основу системи числення, в якій записано число
// в стрічці
// *base* == *0* вказує розглядати число як літерал *C*

Класифікація символів, файл <cctype>

У файлі <cctype>[3] знаходиться наступні функції[4]:

- `int isalnum ( int c );` — перевіряє, чи є символ літерою або цифрою.
- `int isalpha ( int c );` — перевіряє, чи є символ літерою.
- `int islower ( int c );` — перевіряє, чи є символ малою літерою.
- `int isupper ( int c );` — перевіряє, чи є символ великою літерою.
- `int isdigit ( int c );` — перевіряє, чи є символ десятковою цифрою.
- `int isxdigit ( int c );` — перевіряє, чи символ є шістнадцятковою літерою.
- `int isprint ( int c );` — перевіряє, чи є символ видимим при виводі.
- `int ispunct ( int c );` — перевіряє, чи символ є символом пунктуації. (За замовчуванням – всі, що не є isprint, (крім пробілу) але не є isalnum.)
- `int isspace ( int c );` — перевіряє, чи символ є різновидом пробілу (пробілом, табуляцією, символом нового рядка, тощо).

[3] Два c — не опечатка!

[4] Список, як і попередній, не є вичерпним

Перетворення символів, файл <cctype>

Також у <cctype> є пара функцій для перетворення символів:

- `int tolower ( int c );` — якщо символ є великою літерою, перетворює її на малу, інакше не змінює.
- `int toupper ( int c );` — якщо символ є малою літерою, перетворює її на велику, інакше не змінює.

SSO – Short String Optimization

- Сучасні реалізації C++ використовують SSO.
  - Станом на 2022, GCC та MSVC дозволяють стрічці мати до 15 символів, перш ніж здійснюють динамічну алокацію, за розміру об'єкту 32 байти.
  - Clang – до 23, за розміру об'єкта 24 байти! Подробиці див. `https://joellaity.com/2020/01/31/string.html` – "libc++'s implementation of std::string".
- Детальний опис реалізації SSO в MSVC: `https://devblogs.microsoft.com/oldnewthing/20230803-00/?p=108532` – "Inside STL: The string".
- Див. також: `https://www.cppstories.com/2022/sso-cpp20-checks/`.

Посилання

- What is string interning? – есей про цікаву оптимізацію.

У файлі <sstream> оголошено спеціальні потоки, котрі, фактично, є стрічками в пам'яті: stringstream, istringstream, ostringstream. Вони є потомками iostream, istream чи ostream, відповідно.

Працювати з ними можна, як із звичайними потоками, але за допомогою методу str() можна звертатися до них як до стрічок.

Приклад — вивід нашого вектора так, щоб враховувалося передане потоку значення setw():

```
ostream &operator<<(ostream &out,const OurVector& o)
{
    stringstream  tstr;
    tstr << "(" << o.get_x() << "," << o.get_y() << ")";
    // Підготували вивід одним" шматком"
    out << tstr.str();
    // Вивели його однією операцією
    return out;
}
```

Вміст istringstream можна передавати конструктору:

```
istringstream  st("12:44:53");
int h,m,s;
st >> h;
```

- C++20 додав переміщення з std::string в stringstream.

Посилання

- `https://quuxplusone.github.io/blog/2021/04/17/pathological-string-appends/` — як складно враховувати все і зразу...



-
-